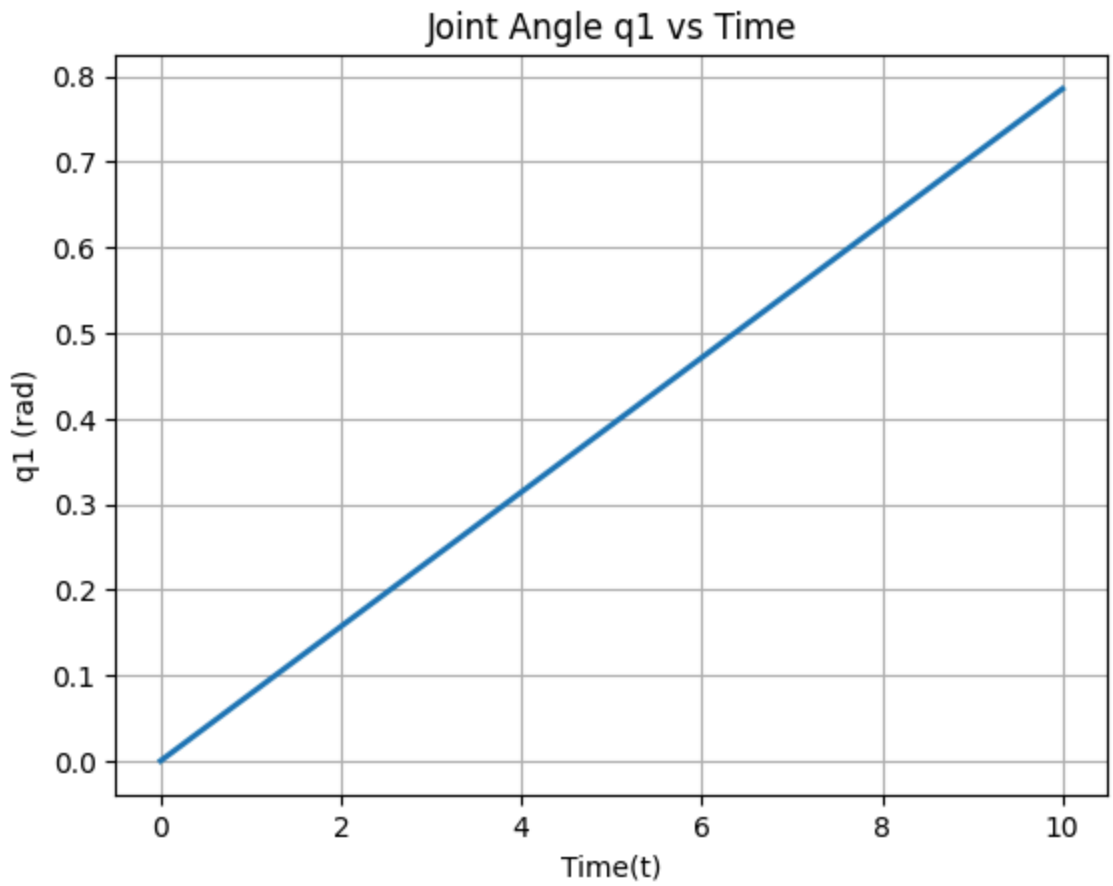
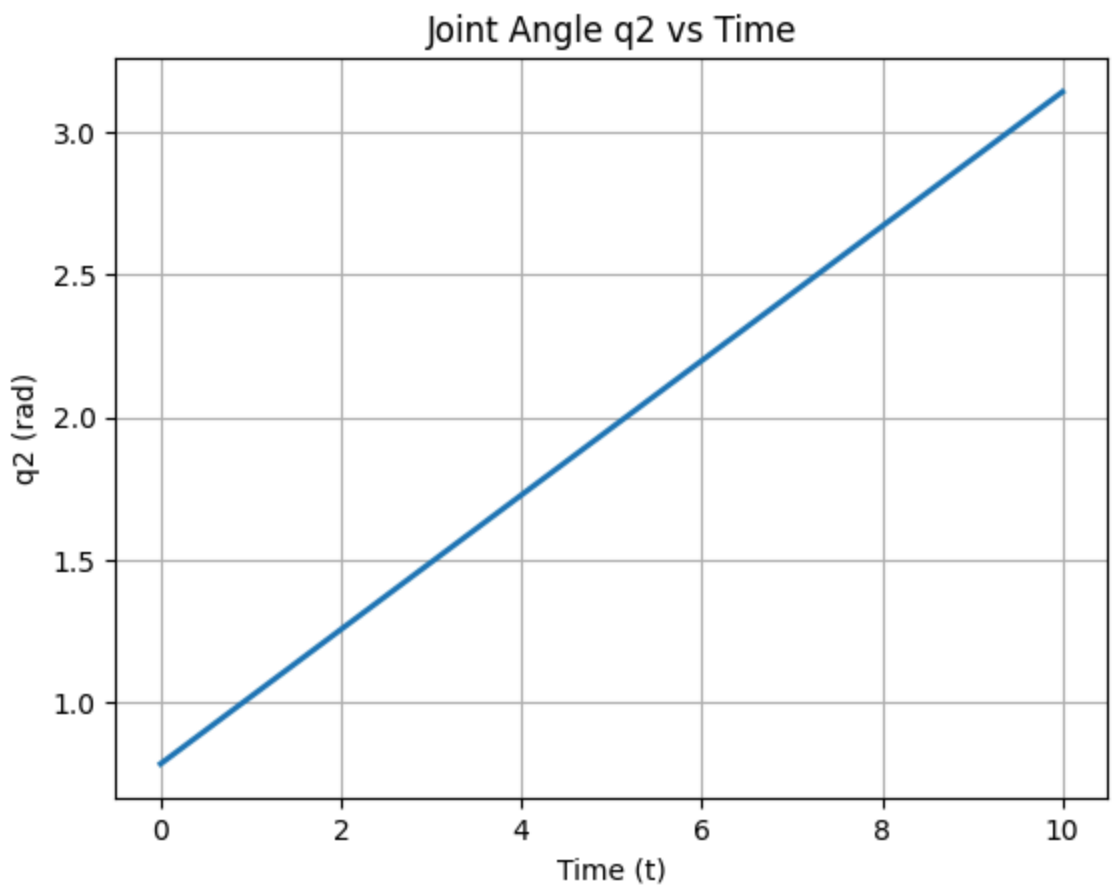
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

def linear_trajectory(q1_start,q2_start,q1_end,q2_end,T,N):
  t = np.linspace(0, T, N)
  q1=q1_start + (t/T)*(q1_end-q1_start)
  q2=q2_start + (t/T)*(q2_end-q2_start)
  #For q1(t)
  plt.figure()
  plt.plot(t,q1,linewidth=2)
  plt.grid(True)
  plt.xlabel("Time(t)")
  plt.ylabel("q1 (rad)")
  plt.title("Joint Angle q1 vs Time")
  plt.show()
  #For q2(t)
  plt.figure()
  plt.plot(t,q2,linewidth=2)
  plt.grid(True)
  plt.xlabel("Time (t)")
  plt.ylabel("q2 (rad)")
  plt.title("Joint Angle q2 vs Time")
  plt.show()
```

In [2]:
```
q1_start = 0
q2_start = np.pi/4
q1_end = np.pi/4
q2_end = np.pi
linear_trajectory(q1_start,q2_start,q1_end,q2_end,10,100)
```

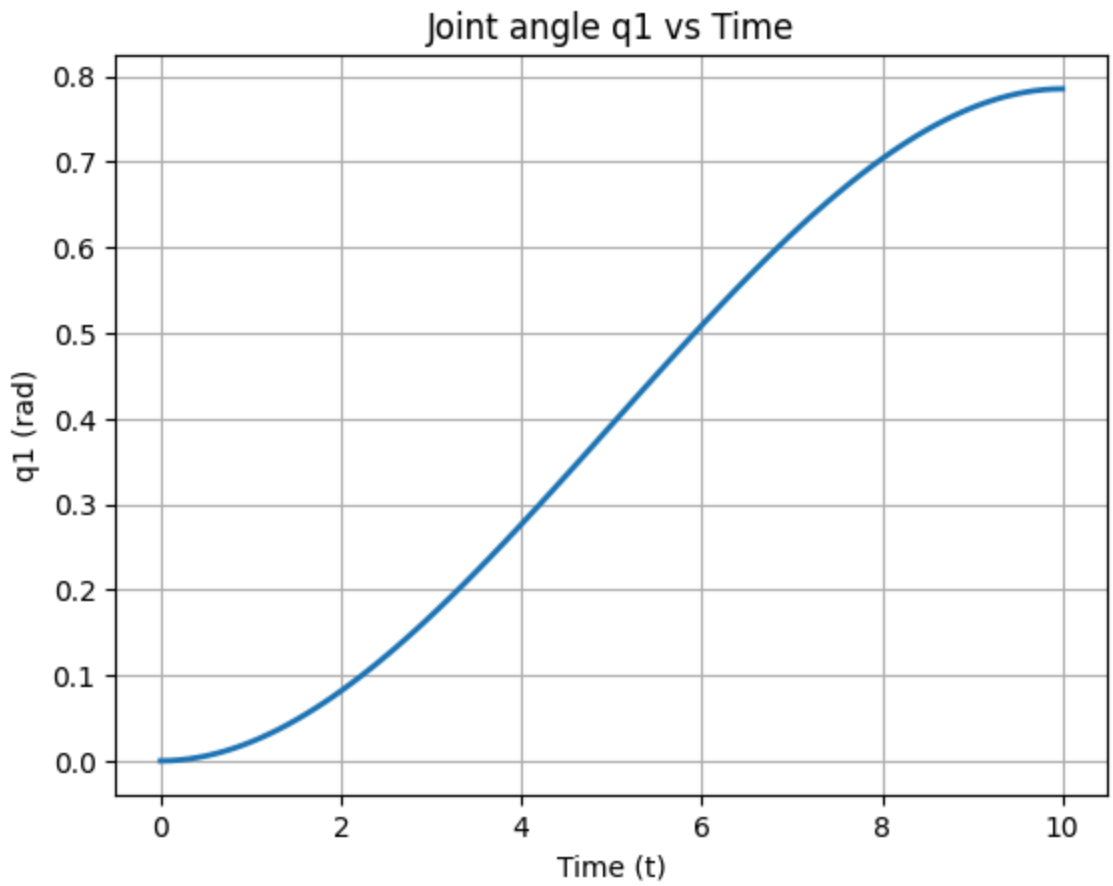

Joint Angle q1 vs Time
q1 (rad)
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0
2
4
6
8
10
Time(t)

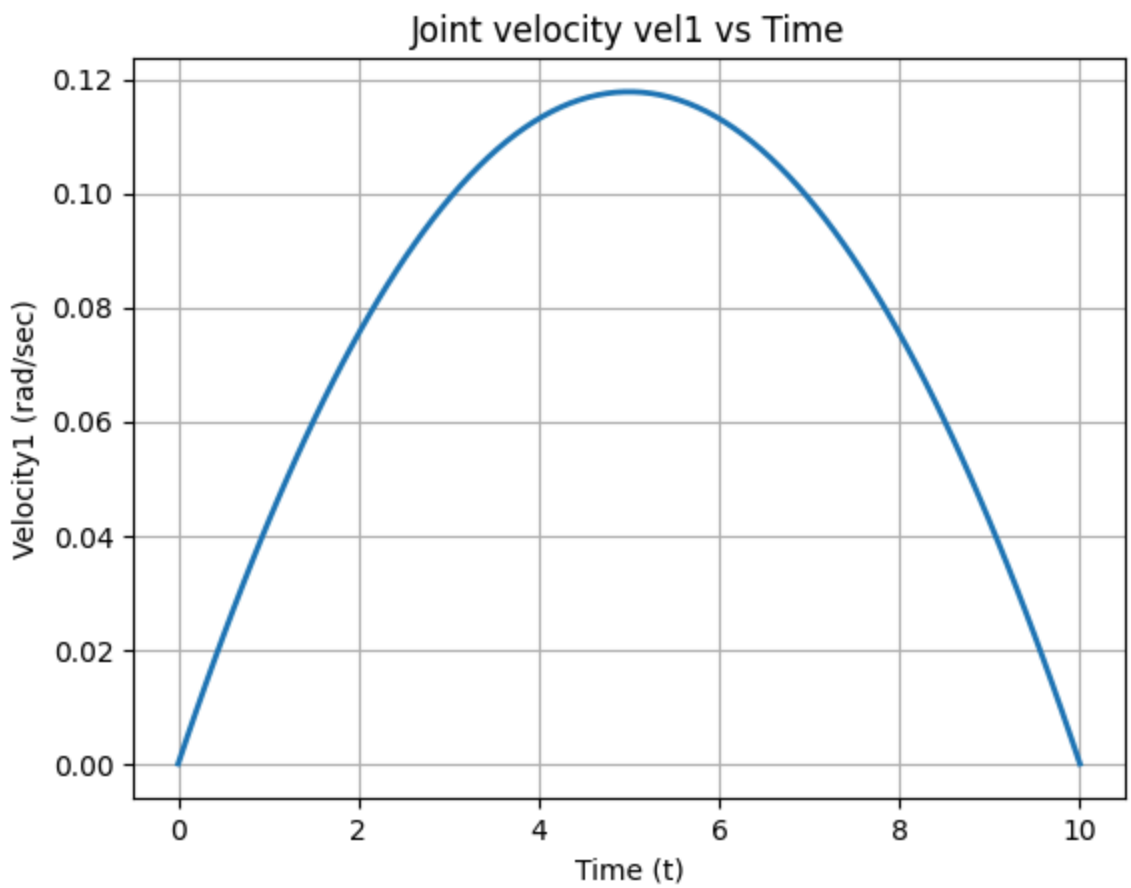

Joint Angle q2 vs Time
q2 (rad)
1.0
1.5
2.0
2.5
3.0
0
2
4
6
8
10
Time (t)

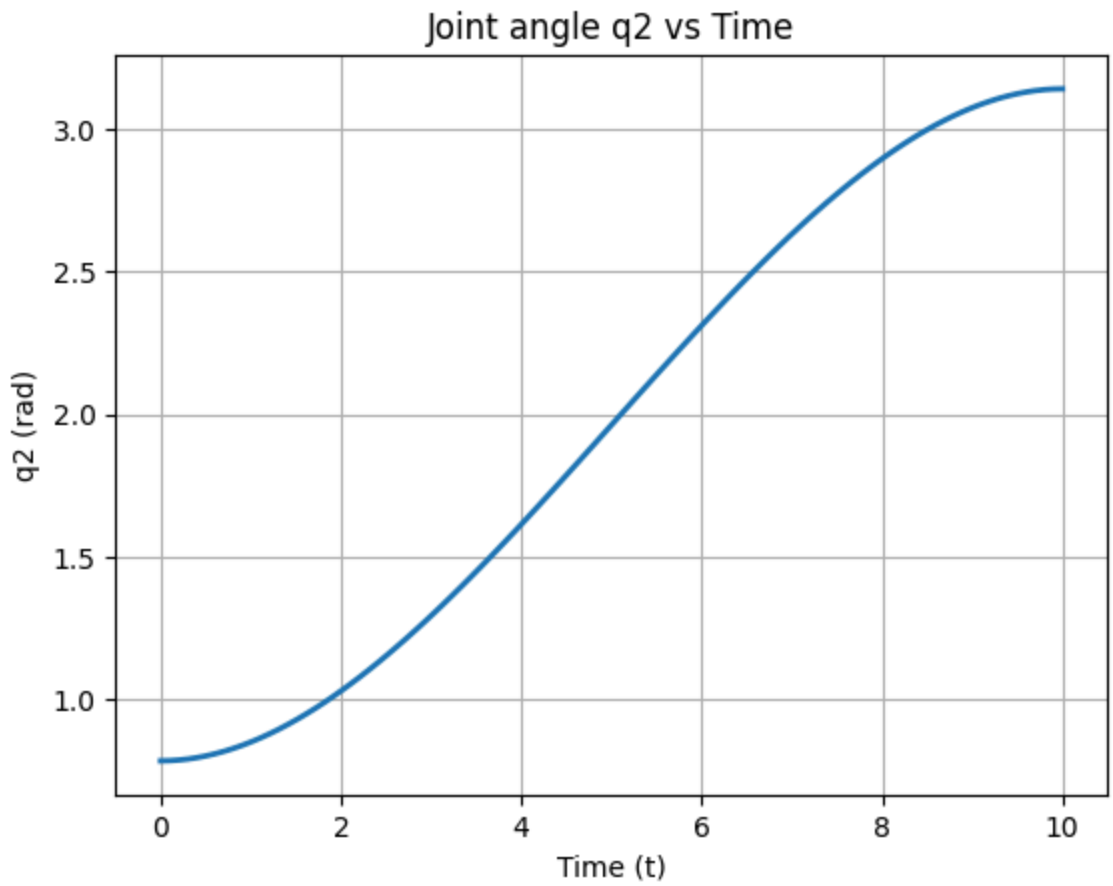
In [3]:
```
def cubic_trajectory(q0,qf,T,N):
  t = np.linspace(0, T, N)
  A = np.array([[1, 0, 0, 0],
                [1, T, T**2, T**3],
                [0, 1, 0, 0],
                [0, 1, 2*T, 3*T**2],
                ])
  b = np.array([q0, qf, 0, 0])
  coeffs = np.linalg.solve(A, b)
  q = coeffs[0] + coeffs[1] * t + coeffs[2] * t ** 2 + coeffs[3] * t ** 3
  vel = coeffs[1] + 2 * coeffs[2] * t + 3 * coeffs[3] * t ** 2
  acc = 2 * coeffs[2] + 6 * coeffs[3] * t

  return t, q, vel, acc
```

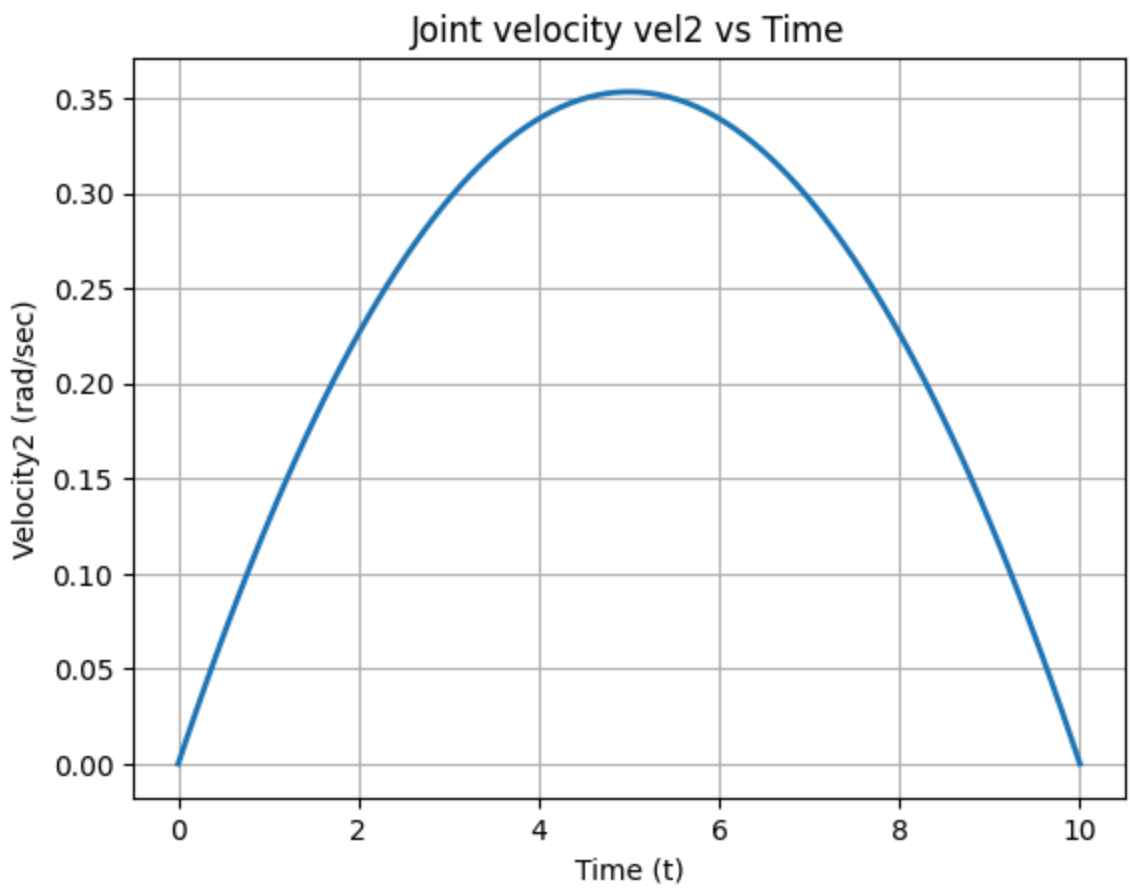
In [4]:
```
q1_start = 0
q2_start = np.pi/4
q1_end = np.pi/4
q2_end = np.pi
t, q1, vel1, acc1 = cubic_trajectory(q1_start, q1_end, 10, 100)
t, q2, vel2, acc2 = cubic_trajectory(q2_start, q2_end, 10, 100)
```

In [6]:
```
#For q1(t)
plt.figure()
plt.plot(t,q1,linewidth=2)
plt.grid(True)
plt.xlabel("Time (t)")
plt.ylabel("q1 (rad)")
plt.title("Joint angle q1 vs Time")
plt.show()

plt.figure()
plt.plot(t,vel1,linewidth=2)
plt.grid(True)
plt.xlabel("Time (t)")
plt.ylabel("Velocity1 (rad/sec)")
plt.title("Joint velocity vel1 vs Time")
plt.show()
#For q2(t)
plt.figure()
plt.plot(t,q2,linewidth=2)
plt.grid(True)
plt.xlabel("Time (t)")
plt.ylabel("q2 (rad)")
plt.title("Joint angle q2 vs Time")
plt.show()

plt.figure()
plt.plot(t,vel2,linewidth=2)
plt.grid(True)
plt.xlabel("Time (t)")
plt.ylabel("Velocity2 (rad/sec)")
plt.title("Joint velocity vel2 vs Time")
plt.show()
```


Joint angle q1 vs Time
q1 (rad)
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0
2
4
6
8
10
Time (t)


Joint velocity vel1 vs Time
Velocity1 (rad/sec)
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0
2
4
6
8
10
Time (t)


Joint angle q2 vs Time
q2 (rad)
3.0
2.5
2.0
1.5
1.0
0
2
4
6
8
10
Time (t)


Joint velocity vel2 vs Time
Velocity2 (rad/sec)
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
2
4
6
8
10
Time (t)

Task 3:

1. Linear trajectories directly interpolate between the start and end joint angles, leading to constant velocity but discontinuous acceleration at the start and end of the motion, whereas, smooth trajectories enforce zero velocities at the start and end of the motion, thereby, producing continuous position, velocity, and acceleration profiles.
2. Smoothness is important for real robots as jerky movements can cause damage to the robot or to the surrounding it interacts with. Smooth trajectories lead to safe, and efficient movement of the robot as it avoids sudden change in velocity and acceleration.